

Document Reporting - PoC Service lit d'urgence

Document Reporting - PoC Service lit d'urgence.....	1
Introduction.....	2
Architecture de la PoC.....	3
Architecture Fonctionnelle.....	3
Architecture Technique.....	3
Architecture Technologique.....	4
Respect Principes et normes.....	5
Résultats de la PoC.....	6
Réutilisabilité.....	7



Introduction

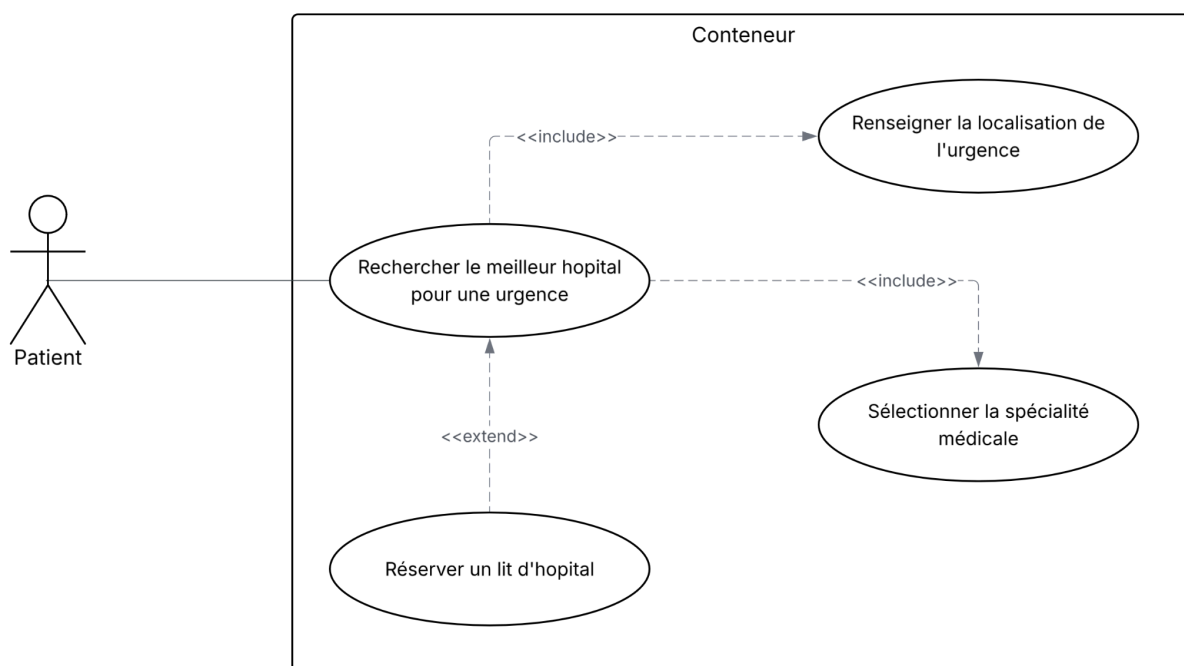
Dans le cadre du projet Medhead, le développement d'une PoC a été initié afin d'évaluer la capacité de la solution à répondre aux besoins d'un système d'intervention d'urgence en temps réel.

Les différents critères de succès de la PoC sont : acheminer plus de 90% des cas d'urgence vers l'hôpital compétent le plus proche, et réduire le temps moyen d'une urgence de 18,25 min à 12 min.

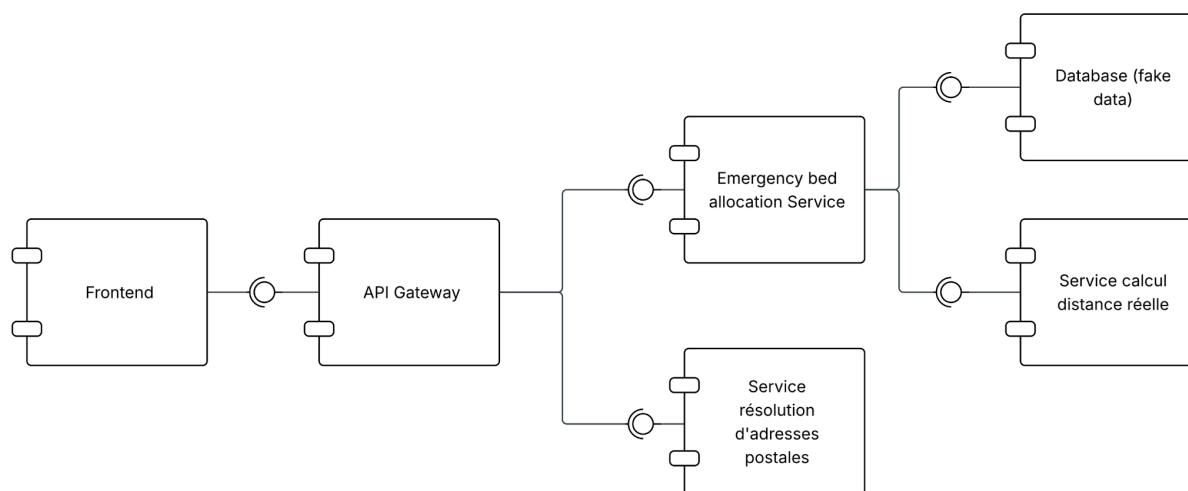
Objectif de performance attendu : obtenir un temps de réponse inférieur à 200 ms avec une charge de 800 requêtes par seconde.

Architecture de la PoC

Architecture Fonctionnelle



Architecture Technique



Architecture Technologique

Frontend :

Vue.js 3 : <https://vuejs.org/> => Réactif et performant, idéal pour un développement rapide de PoC

Emergency bed service :

Spring Boot : <https://spring.io/projects/spring-boot> => Robuste, s'intègre bien dans une architecture micro-services, et rentre dans le critère de la technologie Java imposée.

Base de données :

Postgresql : <https://www.postgresql.org/> => base relationnelle fiable

Service de résolution d'adresses :

Api Gouvernement Français : <https://data.geopf.fr/geocodage> => service souverain et autohébergeable

Service de calcul distance réelle :

OSRM : <https://project-osrm.org/> => open source et autohébergeable

Tests :

- E2E : Playwright <https://playwright.dev/>
- Stress : Jmeter <https://jmeter.apache.org/>
- Unitaire/intégration : JUnit + Testcontainers <https://junit.org/>
<https://testcontainers.com/>

=> Répond à l'exigence de valider entièrement la PoC avec une pyramide de tests complets (unitaires, intégration, stress, e2e)

Respect Principes et normes

RPGD

Dans le cadre de la PoC les seules données du patient recueillit sont son adresse. Cette donnée n'est pas stockée, et l'utilisateur est informé de l'utilisation de cette donnée pour trouver l'hôpital le plus proche de sa position.

De plus, la PoC utilise des données factices pour être isolée et respecter les données réelles des patients (Principe C4).

Intégration et livraison continue

Dans le cadre de la PoC une pipeline d'intégration continue à été développé, elle permet de déclencher la pyramide de tests (unitaire, intégration, stress, e2e) afin de valider les principes B3 et B4.

Cette pipeline pourra être améliorée dans l'architecture cible afin d'automatiser le déploiement.

Critères technologiques et architecturaux

Les contraintes technologiques imposées par le consortium ont été respectées, une api REST en java spring, et une interface en vue.js. De plus, le composant de recherche d'hôpital développé lors de la PoC vise à s'intégrer parfaitement dans une architecture micro-service, afin de garantir une responsabilité unique qui est découplé des autres services hospitaliers.

Résultats de la PoC

Performances et temps de réponses

Temps de réponses de l'api de la PoC sous très fort trafic (800 requêtes en 1 seconde)

Requests	Executions			Response Times (ms)						
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct
Total	1600	0	0.00%	1249.35	93	2652	1361.00	1621.00	1665.95	2402.48
/api/hospitals/findNearest	800	0	0.00%	1206.11	93	1712	1361.00	1631.90	1651.95	1682.99
/api/speciality-groups	800	0	0.00%	1292.60	136	2652	1361.50	1533.90	2173.75	2582.00

On se rend compte que les temps de réponses sont bien loin de ceux attendus

Réutilisabilité

En accord avec la demande du comité d'architecture, la PoC a été conçue afin de proposer des Solution Building Blocks utilisables pour le développement de la solution cible :

- Le socle technique (Spring Boot/ Vue js) pourra servir de modèle pour les prochains micro-services
- La pipeline automatisé pourra être amélioré pour gérer les déploiement
- Les conventions (versionning, nommage) utilisés lors de la PoC pourront être maintenue lors du développement de la cible

Conclusion

Le développement de cette PoC a permis d'évaluer la capacité du consortium Medhead à répondre aux besoins du système d'intervention d'urgence en temps réel.

L'objectif de performance pour (100ms sous 800 req/s) a échoué, en raison de limitation d'api tierce, cependant la cause est isolée et une solution existe (auto-hébergement). Sur le plan fonctionnel, technologique et réglementaire, les exigences ont été respectées.

Ce projet fournit une validation des choix architecturaux, et met à disposition des équipes une base solide pour le développement de l'architecture cible.